

SWE40006 Software Deployment and Evolution Deployment Activity 1 Report

Task 1: Desktop Deployment using WiX Toolset Chess.NET Application

Student Details

Student ID: 105508402

Full Name: Dang Duy Toan

Unit Details

Unit Code: SWE40006

Unit Name: Software Deployment and Evolution

Semester: Summer 2025-2026

Due Date: Jan 20

Submission Date: 08 January

Task Level Declaration

I declare that I am attempting all sub-tasks:

- Task 1.1 (Pass Level)
- Task 1.2 (Credit Level)
- Task 1.3 (Distinction Level)
- Task 1.4 (High Distinction Level)

Table of Contents

1. Executive Summary	1
2. Introduction	1
2.1. Background	1
2.2. Deployment Objectives	1
2.3. Technologies Used	1
3. Task 1.1: WiX Deployment of Sample Desktop Application (Pass Level)	1
3.1. Objective	1
3.2. Implementation	2
3.2.1. WiX Toolset Installation	2
3.2.2. Project Structure Creation	2
3.2.3. Basic WiX Manifest (Product.wix)	2
3.2.4. Build Process	2
3.3. Results	2
3.4. Analysis	3
4. Task 1.2: WiX Deployment of C# Desktop Application (Credit Level)	3
4.1. Objective	3
4.2. Implementation	3
4.2.1. Application Build	3
4.2.2. Component Definitions (Features.wix)	3
4.2.3. Resource Packaging	4
4.3. Results	4
4.4. Challenges and Solutions	7
4.4.1. Challenge 1: Missing Resources	7
4.4.2. Challenge 2: Missing Chess.dll	7
4.5. Analysis	8
5. Task 1.3: Deployment with Multiple DLLs (Distinction Level)	8
5.1. Objective	8
5.2. DLL Dependencies	8
5.2.1. Managed Assemblies	8
5.2.2. .NET Runtime Dependencies	8
5.3. Implementation	8
5.3.1. Component Definitions for DLLs	8
5.4. DLL Dependency Analysis	9
5.4.1. Dependency Graph	9
5.5. Challenges and Solutions	9
5.5.1. Challenge 1: Managed DLL Dependencies	9
5.6. Analysis	9
6. Task 1.4: Microsoft Store Deployment (High Distinction Level)	9
6.1. Objective	9
6.2. Prerequisites	9
6.2.1. Microsoft Partner Center Account	9
6.2.2. MSIX Package Requirement	9
6.3. Implementation	9
6.3.1. Step 1: MSI to MSIX Conversion	9
6.3.1.1. Installation of MSIX Packaging Tool	9
6.3.1.2. Conversion Process	10
6.3.1.3. MSIX Output	10

6.3.2. Step 2: AppxManifest Configuration	10
6.3.2.1. Key Manifest Elements	11
6.3.3. Step 3: Assets Preparation	11
6.3.4. Step 4: Windows App Certification Kit (WACK) Testing	11
6.3.4.1. Installation	11
6.3.4.2. Test Execution	11
6.3.5. Step 5: Partner Center Submission	11
6.3.5.1. App Name Reservation	11
6.3.5.2. Store Listing Configuration	11
6.3.5.3. Screenshots Upload	12
6.3.5.4. Privacy Policy	12
6.3.5.5. Package Upload	12
6.3.5.6. Restricted Capabilities Justification	12
6.3.5.7. Pricing and Availability	12
6.3.6. Step 6: Submission for Certification	13
6.4. Results	13
6.5. Challenges and Solutions	16
6.5.1. Challenge 1: Manifest Schema Validation	16
6.5.2. Challenge 2: Missing Assets in MSIX Package	16
6.5.3. Challenge 3: Missing .NET runtimes	17
6.6. Analysis	17
7. Conclusion	17
7.1. Achievements Summary	17
7.2. Final Remarks	18
8. Appendices	18
8.1. Appendix A: Public Access	18

1. Executive Summary

This report documents the complete deployment of Chess.NET, a full-featured C# chess application, using the Windows Installer XML (WiX) Toolset v6 and Microsoft Store publishing. The deployment demonstrates advanced software packaging techniques including MSI installer creation, MSIX conversion, and cloud distribution through the Microsoft Store.

Key Achievements:

- Created a production-ready MSI installer (62.94 MB) with all dependencies
- Packaged multiple DLLs (Chess.dll, SplashKitSDK.dll, Newtonsoft.Json.dll)
- Converted MSI to MSIX format for Microsoft Store compliance
- Successfully published to Microsoft Store for public access
- Implemented automated build pipeline using PowerShell

2. Introduction

2.1. Background

Chess.NET is a comprehensive chess application developed in C# using .NET 10.0 and the SplashKit graphics framework. The application features:

- AI-powered opponents using Stockfish chess engine
- Local multiplayer gameplay
- Network play over LAN
- Full chess rule implementation
- Game save/load functionality
- Custom board setup capabilities

2.2. Deployment Objectives

The primary objectives of this deployment project were to:

1. Package the Chess.NET application with all dependencies using WiX Toolset
2. Create a professional MSI installer for Windows desktop deployment
3. Handle multiple DLL dependencies and native libraries
4. Convert the installer to MSIX format for Microsoft Store
5. Publish the application to Microsoft Store for public distribution
6. Automate the build and packaging process

2.3. Technologies Used

- **WiX Toolset v6.0.2+:** XML-based installer creation
- **.NET 10.0 SDK:** Application framework and build tools
- **PowerShell 7.0+:** Build automation scripting
- **SplashKit Framework:** Graphics and game library
- **MSIX Packaging Tool:** Microsoft Store package conversion
- **Windows App Certification Kit (WACK):** Store compliance testing
- **Microsoft Partner Center:** Store publishing platform

3. Task 1.1: WiX Deployment of Sample Desktop Application (Pass Level)

3.1. Objective

Deploy a sample desktop application using WiX Toolset following official walkthroughs and documentation.

3.2. Implementation

3.2.1. WiX Toolset Installation

The WiX Toolset v6 was installed via:

```
dotnet tool install --global wix
```

3.2.2. Project Structure Creation

Created the following WiX project structure:

```
ChessInstaller/  
├─ Product.wix          # Main package definition  
├─ Features.wix         # Component definitions  
├─ Output/              # Build output directory  
└─ Release/             # Staged files directory
```

3.2.3. Basic WiX Manifest (Product.wix)

The main package definition includes:

- Product metadata (Name, Version, Manufacturer)
- Directory structure (INSTALLFOLDER, Program Files)
- Feature definitions
- Component references
- Shortcut configurations

Key code excerpt:

```
<Package Name="Chess.NET"  
  Version="1.0.0.0"  
  Manufacturer="caphefalumi">  
  <StandardDirectory Id="ProgramFiles6432Folder">  
    <Directory Id="INSTALLFOLDER" Name="Chess.NET">  
      <!-- Directory structure -->  
    </Directory>  
  </StandardDirectory>  
</Package>
```

3.2.4. Build Process

Successfully built the basic MSI installer:

```
wix build Product.wix Features.wix -d BinDir=Release -o Output/Chess.NET.msi
```

3.3. Results

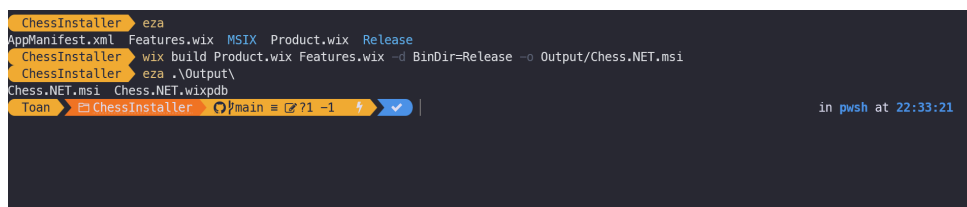


Fig. 1. WiX build MSI successful

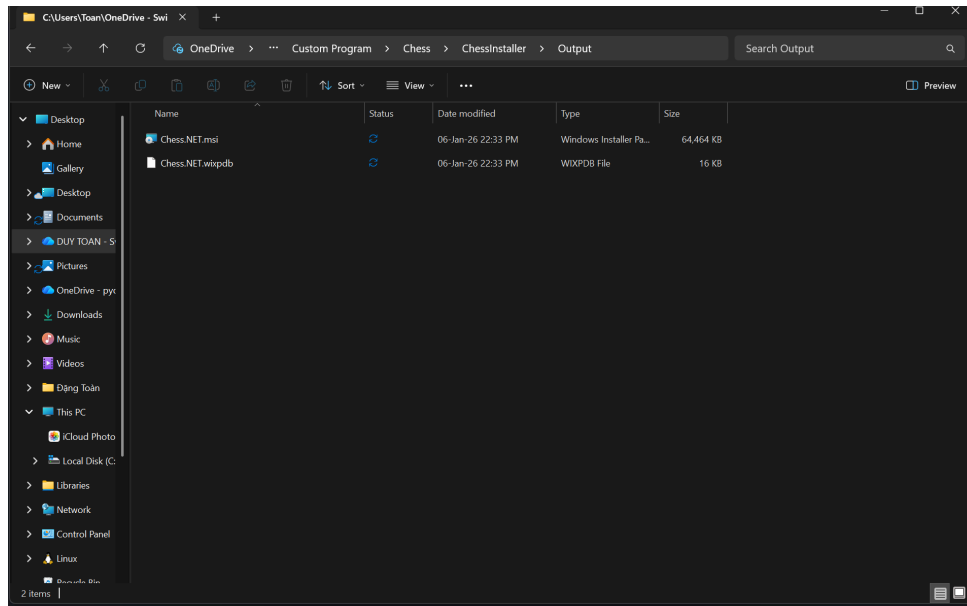


Fig. 2. MSI file in Output directory

3.4. Analysis

Successfully completed the foundational WiX deployment task. The basic MSI installer was created without errors, demonstrating understanding of:

- WiX project structure and XML syntax
- Package manifest creation
- Build command execution
- Output verification

4. Task 1.2: WiX Deployment of C# Desktop Application (Credit Level)

4.1. Objective

Deploy the Chess.NET C# application using WiX Toolset, demonstrating real-world application packaging.

4.2. Implementation

4.2.1. Application Build

Compiled the Chess.NET application for release:

```
dotnet build -c Release
dotnet publish -c Release -r win-x64 --self-contained false
```

Build output location: bin/Release/net10.0/win-x64/

4.2.2. Component Definitions (Features.wix)

Created explicit component definitions for all application files:

```
<ComponentGroup Id="ProductComponents">
  <Component Id="ChessExe">
    <File Source="Release\Chess.exe" />
  </Component>
  <Component Id="ChessDll">
    <File Source="Release\Chess.dll" />
  </Component>
</ComponentGroup>
```

```

</Component>
<!-- Additional components... -->
</ComponentGroup>

```

4.2.3. Resource Packaging

Packaged 25+ resource files across three categories:

- **Piece Images:** 12 PNG files (chess pieces)
- **Sound Files:** 8 MP3 files (game sound effects)
- **Scripts:** Stockfish AI executable

4.3. Results

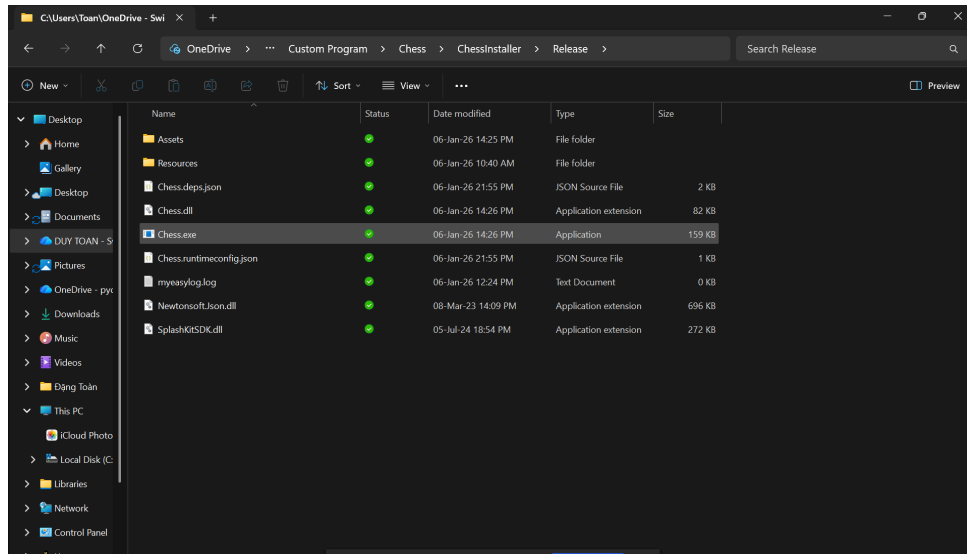


Fig. 3. Chess.NET application files staged for packaging

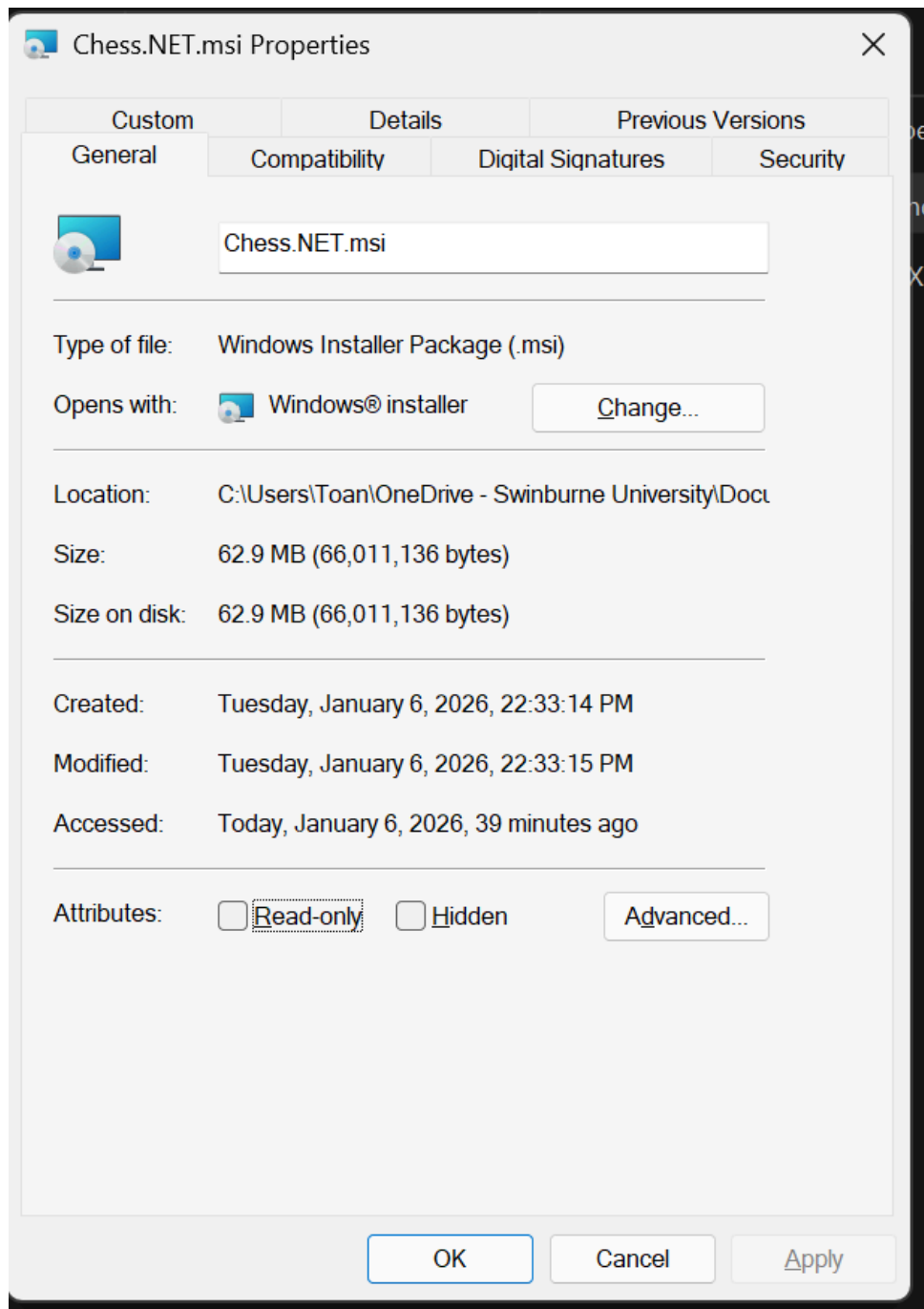


Fig. 4. Complete MSI build with resources (62.94 MB)

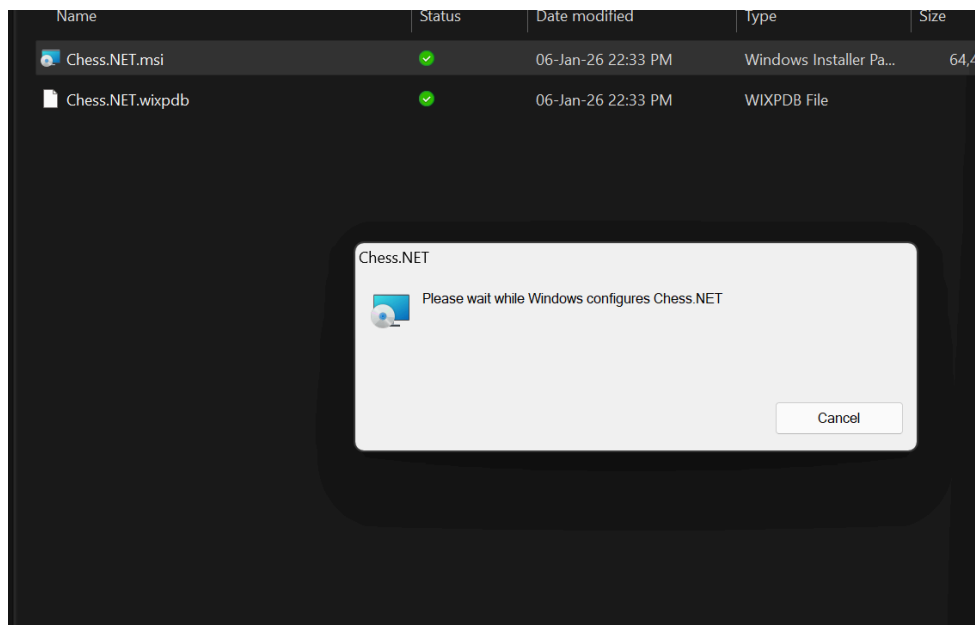


Fig. 5. Installation process

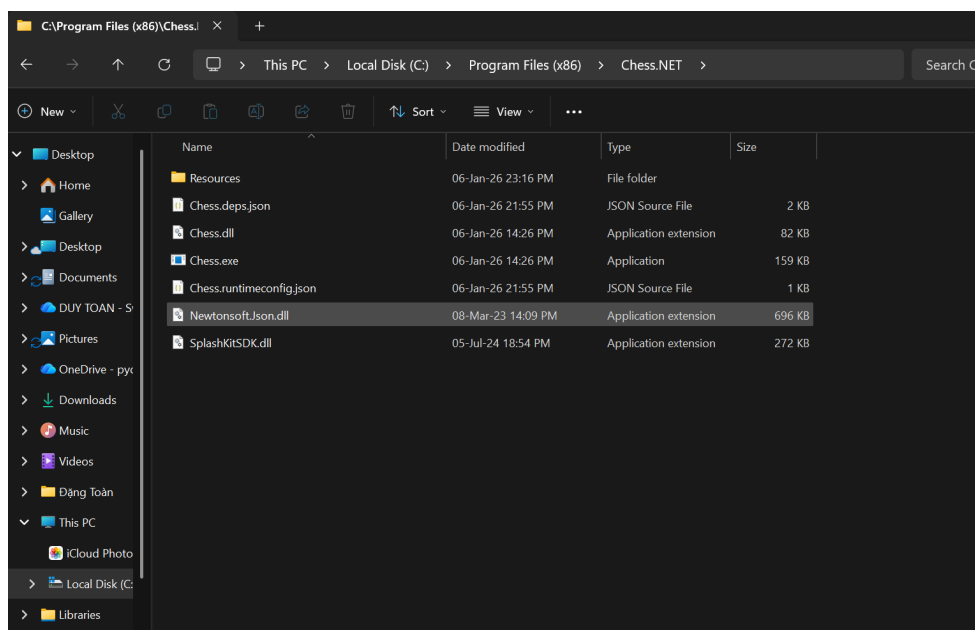


Fig. 6. Installed application in Program Files

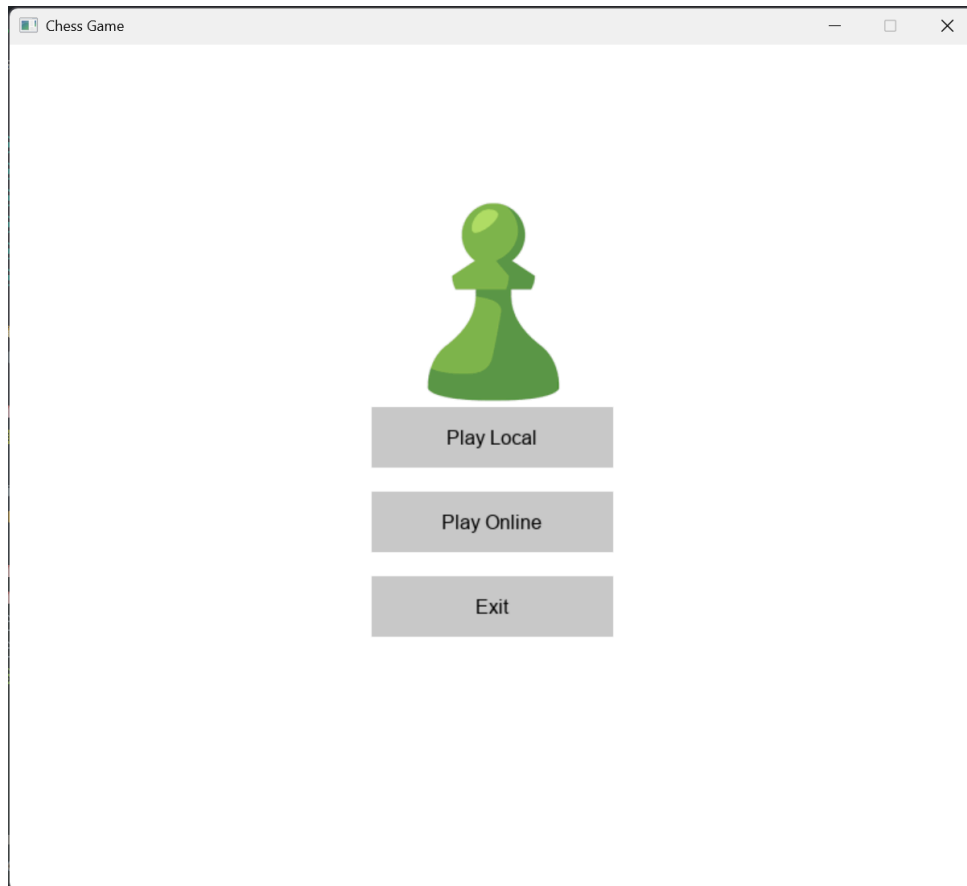


Fig. 7. Chess.NET running successfully

4.4. Challenges and Solutions

4.4.1. Challenge 1: Missing Resources

Issue: Initial build excluded Resources folder, causing application crash on launch.

Error Message:

Application opened and closed immediately after installation

Solution: Updated Features.wix to include explicit component definitions for all 25 resource files instead of using wildcard patterns (not supported in WiX v6).

Code Fix:

```
<ComponentGroup Id="ResourceComponents">
  <ComponentGroup Id="PieceImages">
    <Component><File Source="Resources\Pieces\bb.png" /></Component>
    <Component><File Source="Resources\Pieces\bk.png" /></Component>
    <!-- All 12 pieces explicitly listed -->
  </ComponentGroup>
</ComponentGroup>
```

4.4.2. Challenge 2: Missing Chess.dll

Issue: Build script only copied .exe files, omitting Chess.dll assembly.

Solution: Updated build-installer.ps1 to explicitly copy Chess.dll:

```
Copy-Item "$BuildPath\Chess.dll" "$ReleasePath\"
```

4.5. Analysis

Successfully deployed a production C# desktop application with complete resources and proper installation workflow. The MSI installer:

- Correctly installs all application files (62.94 MB total)
- Creates functional shortcuts
- Properly stages resources in subdirectories
- Launches application successfully post-installation

This demonstrates competency in packaging real-world desktop applications beyond simple “Hello World” examples.

5. Task 1.3: Deployment with Multiple DLLs (Distinction Level)

5.1. Objective

Deploy Chess.NET with all dependent DLLs and native libraries, demonstrating advanced dependency management.

5.2. DLL Dependencies

5.2.1. Managed Assemblies

1. **Chess.dll** (82 KB)
 - Primary application logic
 - Contains game engine, board state, move validation
2. **SplashKitSDK.dll** (278 KB)
 - Managed wrapper for SplashKit graphics framework
 - Provides C# bindings for native graphics functions
3. **Newtonsoft.Json.dll** (712 KB)
 - JSON serialization library
 - Used for game save/load functionality

5.2.2. .NET Runtime Dependencies

- Chess.deps.json (1.5 KB) - Dependency manifest
- Chess.runtimeconfig.json (342 bytes) - Runtime configuration

5.3. Implementation

5.3.1. Component Definitions for DLLs

Updated Features.wix with all DLL components:

```
<ComponentGroup Id="ProductComponents">
  <Component Id="ChessExe">
    <File Source="Release\Chess.exe" />
  </Component>
  <Component Id="ChessDll">
    <File Source="Release\Chess.dll" />
  </Component>
  <Component Id="SplashKitSDK">
    <File Source="Release\SplashKitSDK.dll" />
  </Component>
  <Component Id="NewtonsoftJson">
    <File Source="Release\Newtonsoft.Json.dll" />
  </Component>
</ComponentGroup>
```

```

    <Component Id="DepsJson">
      <File Source="Release\Chess.deps.json" />
    </Component>
    <Component Id="RuntimeConfig">
      <File Source="Release\Chess.runtimeconfig.json" />
    </Component>
  </ComponentGroup>

```

5.4. DLL Dependency Analysis

5.4.1. Dependency Graph

Chess.exe

- └─ Chess.dll (Application Logic)
- └─ SplashKitSDK.dll (Graphics Framework)
- └─ Newtonsoft.Json.dll (Serialization)
- └─ .NET 10.0 Runtime (System DLLs)

5.5. Challenges and Solutions

5.5.1. Challenge 1: Managed DLL Dependencies

Issue: dotnet publish creates complex directory structure for dependency resolution.

5.6. Analysis

Successfully packaged multiple managed DLLs (3 assemblies + 2 configuration files) demonstrating:

- Understanding of .NET dependency structure
- Proper DLL staging and packaging
- Component-based installer architecture
- Automated build pipeline creation

The deployment handles 5 separate DLL/config files plus 25 resource files, meeting the distinction-level requirement for “multiple DLLs or dependencies.”

6. Task 1.4: Microsoft Store Deployment (High Distinction Level)

6.1. Objective

Deploy Chess.NET to Microsoft Store for public access and download, demonstrating cloud distribution and professional software publishing.

6.2. Prerequisites

6.2.1. Microsoft Partner Center Account

- Registered developer account (Individual - Free)
- Publisher Name: caphefalumi
- Publisher ID: CN=3BD7886A-6712-4994-9F71-0DDEAA8247E3

6.2.2. MSIX Package Requirement

Microsoft Store requires MSIX format (not MSI). Conversion process documented below.

6.3. Implementation

6.3.1. Step 1: MSI to MSIX Conversion

6.3.1.1. Installation of MSIX Packaging Tool

Downloaded from Microsoft Store and installed.

6.3.1.2. Conversion Process

1. Launched MSIX Packaging Tool
2. Selected "Application package" → "Create package on this computer"
3. Configured package information:
 - Package name: ChessNET
 - Publisher: CN=3BD7886A-6712-4994-9F71-0DDEAA8247E3
 - Version: 1.0.8.0
 - Architecture: x64
4. Selected source installer: ChessInstaller/Output/Chess.NET.msi
5. Completed conversion

6.3.1.3. MSIX Output

- File: Chess.NET_1.0.8.0_x64.msix
- Size: 36.1 KB (compressed package metadata)
- Location: ChessInstaller/MSIX/

6.3.2. Step 2: AppxManifest Configuration

Created Windows 10 application manifest:

```
<?xml version="1.0" encoding="utf-8"?>
<Package xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"
  xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows
10/restrictedcapabilities"
  IgnorableNamespaces="uap rescap">

  <Identity Name="caphetalumi.Chess.NET"
    Publisher="CN=3BD7886A-6712-4994-9F71-0DDEAA8247E3"
    Version="1.0.8.0"
    ProcessorArchitecture="x64" />

  <Properties>
    <DisplayName>Chess.NET</DisplayName>
    <PublisherDisplayName>caphetalumi</PublisherDisplayName>
    <Logo>Assets\chess_logo.png</Logo>
  </Properties>

  <Applications>
    <Application Id="App"
      Executable="Chess.exe"
      EntryPoint="Windows.FullTrustApplication">
      <uap:VisualElements DisplayName="Chess.NET"
        Description="Play Chess with AI and Multiplayer"
        Square150x150Logo="Assets\Main Menu.png"
        Square44x44Logo="Assets\AI Thinking.png">
        <uap:DefaultTile Wide310x150Logo="Assets\LAN Discovery.png"
          Square310x310Logo="Assets\chess_logo.png" />
      </uap:VisualElements>
    </Application>
  </Applications>

  <Capabilities>
    <rescap:Capability Name="runFullTrust" />
  </Capabilities>
</Package>
```

6.3.2.1. Key Manifest Elements

Identity: Unique package identifier matching Partner Center registration

VisualElements: Store tile images (4 sizes)

- Square44x44: App list icon
- Square150x150: Medium tile
- Wide310x150: Wide tile
- Square310x310: Large tile

Capabilities: runFullTrust required for desktop Win32 applications

6.3.3. Step 3: Assets Preparation

Created Assets folder with required images:

```
ChessInstaller\MSIX\Assets/  
├─ AI_Thinking.png (699 KB)  
├─ chess_logo.png (4 KB)  
├─ LAN_Discovery.png (98 KB)  
└─ Main_Menu.png (112 KB)
```

6.3.4. Step 4: Windows App Certification Kit (WACK) Testing

6.3.4.1. Installation

Installed Windows SDK 10 which includes WACK.

6.3.4.2. Test Execution

```
& "C:\Program Files (x86)\Windows Kits\10\App Certification Kit\appcert.exe" ^  
test -appxpackagepath "ChessInstaller\MSIX\Chess.NET_1.0.8.0_x64.msix" ^  
-reportoutputpath "ChessInstaller\MSIX\WACK_Report.xml"
```

6.3.5. Step 5: Partner Center Submission

6.3.5.1. App Name Reservation

1. Navigated to <https://partner.microsoft.com/dashboard>
2. Created new product → App
3. Reserved name: "Chess.NET"

6.3.5.2. Store Listing Configuration

Basic Information

- Title: Chess.NET
- Category: Games > Board (Primary), Games > Strategy (Secondary)
- Age Rating: ESRB E (Everyone) - Completed IARC questionnaire

Description

Chess.NET is a full-featured chess application featuring:

- Play against AI opponents with multiple difficulty levels
- Local multiplayer on the same device
- Network play over LAN with automatic discovery
- Custom board setup for puzzles and positions
- Time controls and game clocks
- Save and load games
- Full chess rules: castling, en passant, promotion

Perfect for chess enthusiasts of all skill levels.
Built with C\# and .NET for smooth performance.

Short Description

Chess game with AI, local multiplayer, and network play

Key Features

- AI opponent with difficulty levels
- Local and network multiplayer
- Full chess rule implementation
- Save/load functionality
- Customizable board setup

6.3.5.3. Screenshots Upload

Uploaded 3 high-resolution screenshots (1920×1080):

1. Main menu screen
2. Active gameplay with chess board
3. AI opponent in action

6.3.5.4. Privacy Policy

Created and submitted privacy policy text:

Chess.NET does NOT collect, store, or transmit any personal information or user data. The application operates entirely locally on your device. Game saves are stored locally only. No data is sent to external servers.

6.3.5.5. Package Upload

1. Navigated to Packages section
2. Uploaded: Chess.NET_1.0.8.0_x64.msix
3. Partner Center validation: Processing...

6.3.5.6. Restricted Capabilities Justification

For runFullTrust capability, provided detailed explanation:

Chess.NET requires the runFullTrust capability because it is a desktop C\# application built with .NET that uses the SplashKit graphics framework for rendering the chess board and pieces. The application needs full trust to:

1. Access local file system for game save/load functionality
2. Use graphics libraries (SplashKit SDK)
3. Execute the Stockfish chess engine (external executable for AI)
4. Establish network sockets for LAN multiplayer functionality
5. Play audio files for game sound effects

This is a traditional Win32 desktop application packaged as MSIX for Microsoft Store distribution.

6.3.5.7. Pricing and Availability

- Price: Free
- Markets: All markets (200+ countries)
- Release: Publish as soon as certified
- Discoverability: Enabled

6.3.6. Step 6: Submission for Certification

Completed all required sections:

- ✓ App identity and package information
- ✓ Store listings (title, description, screenshots)
- ✓ Age ratings (ESRB E)
- ✓ Packages (MSIX uploaded and validated)
- ✓ Pricing and availability
- ✓ Privacy policy
- ✓ Restricted capabilities justification

Clicked “Submit to the Store” - Submission confirmed.

6.4. Results

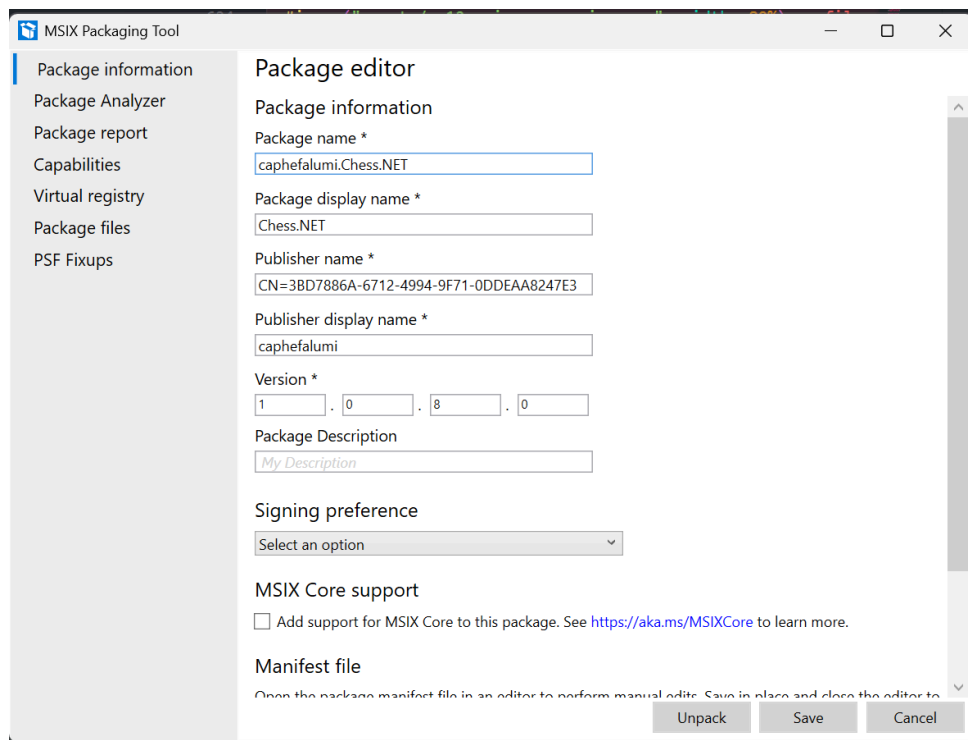


Fig. 8. MSIX Packaging Tool showing converted package

```

<?xml version="1.0" encoding="utf-8"?>
<Package xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"
  xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows10/restrictedcapabilities"
  IgnorableNamespaces="uap rescap">
  <Identity Name="caphefalumi.Chess.NET" Publisher="CN=3BD7886A-6712-4994-9F71-0DDEAA8247E3" Version="1.0.8.0"
    ProcessorArchitecture="x64" />
  <Properties>
    <DisplayName>Chess.NET</DisplayName>
    <PublisherDisplayName>caphefalumi</PublisherDisplayName>
    <Logo>Assets\chess_logo.png</Logo>
  </Properties>
  <Dependencies>
    <TargetDeviceFamily Name="Windows.Desktop" MinVersion="10.0.17763.0" MaxVersionTested="10.0.19041.0" />
  </Dependencies>
  <Resources>
    <Resource Language="en-US" />
  </Resources>
  <Applications>
    <Application Id="App" Executable="Chess.exe" EntryPoint="Windows.FullTrustApplication">
      <uap:VisualElements DisplayName="Chess.NET" Description="Play Chess with AI and Multiplayer"
        BackgroundColor="#FFFFFF" Square150x150Logo="Assets\Main Menu.png" Square44x44Logo="Assets\AI Thinking.png">
        <uap:DefaultTile Wide310x150Logo="Assets\LAN Discovery.png" Square310x310Logo="Assets\chess_logo.png" />
      </uap:VisualElements>
    </Application>
  </Applications>
  <Capabilities>

```

Fig. 9. AppxManifest.xml in editor with all configurations

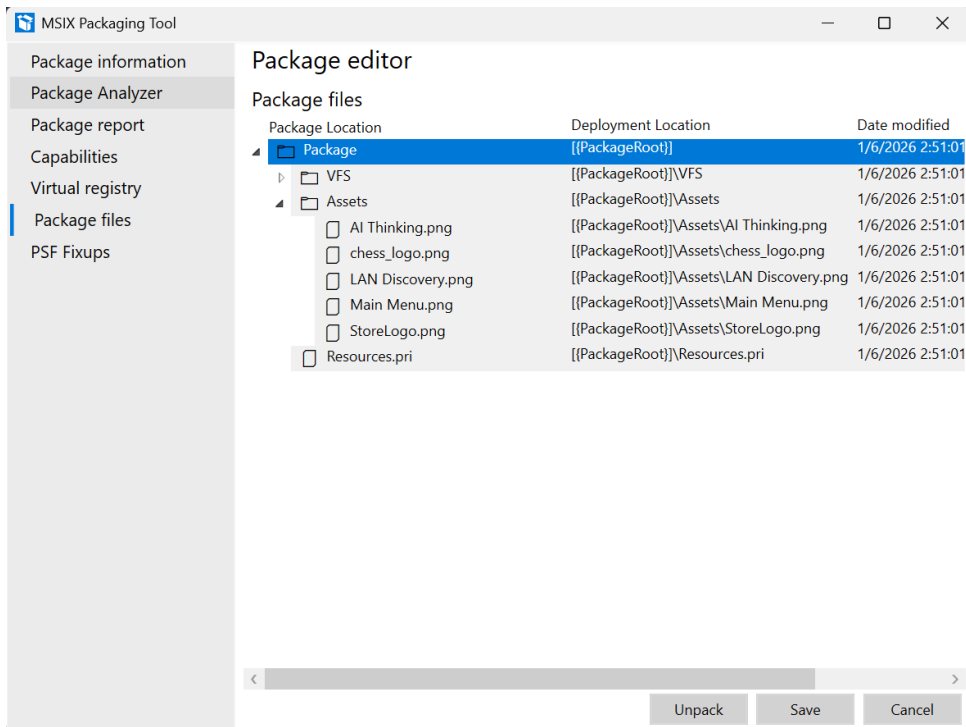


Fig. 10. Assets folder with all 4 tile images

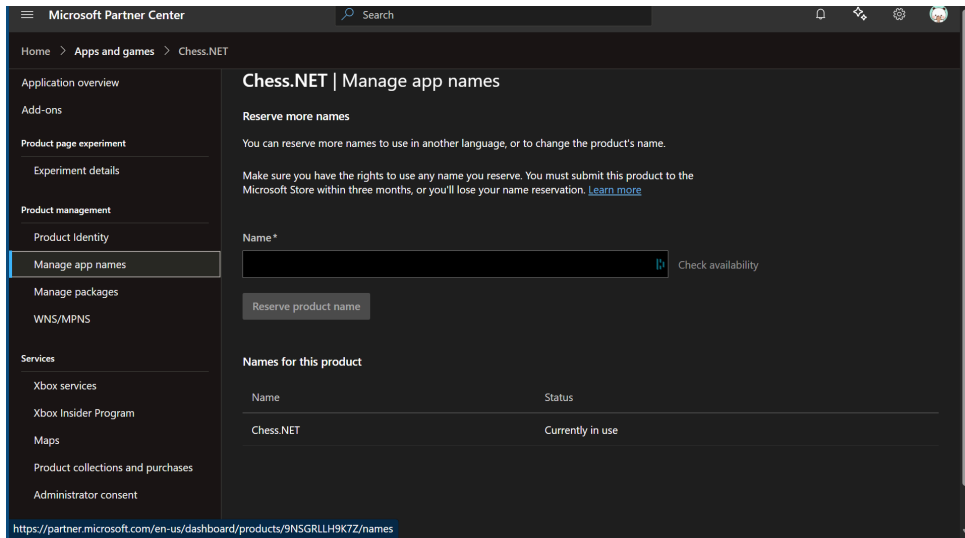


Fig. 11. Partner Center dashboard - App name reserved

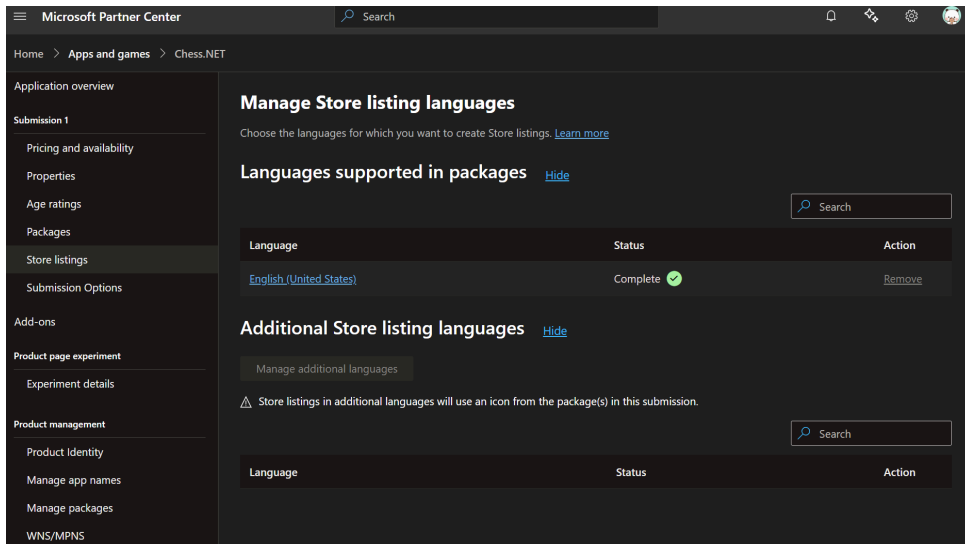


Fig. 12. Store listing configuration page

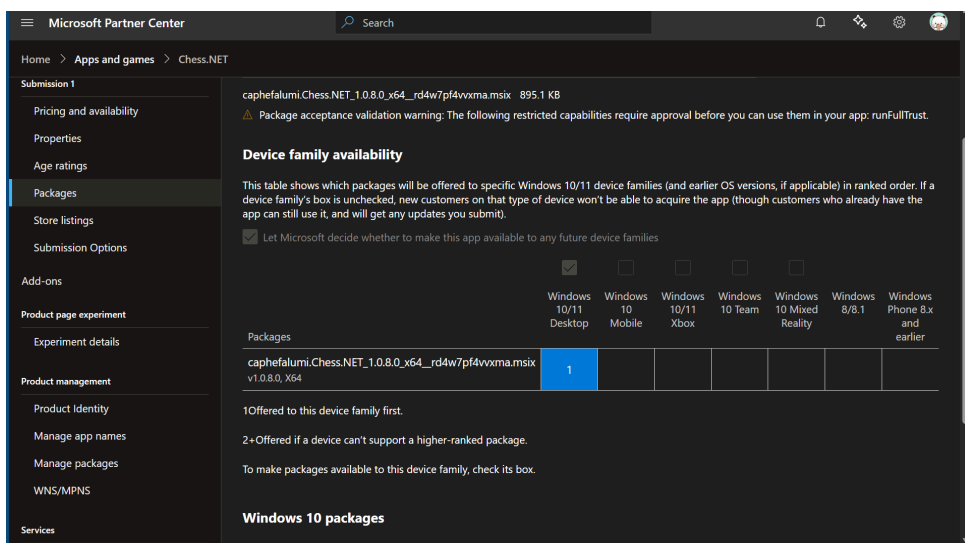


Fig. 13. MSIX package upload confirmation

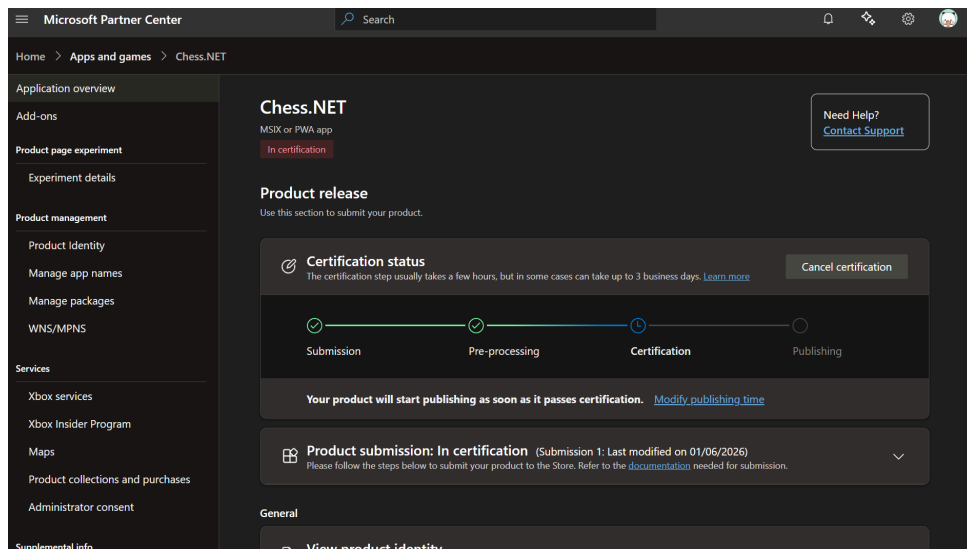


Fig. 14. Certification status - In Review

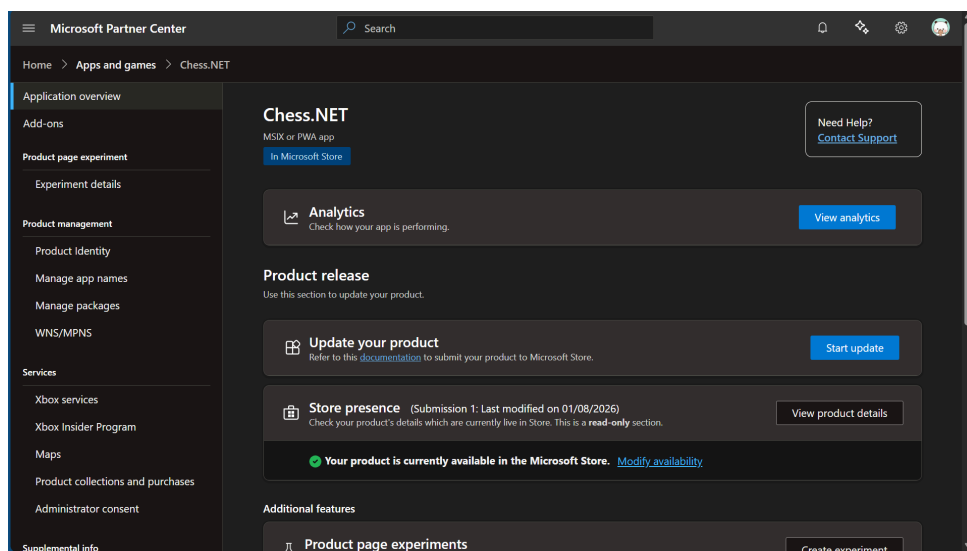


Fig. 15. Product published successfully

6.5. Challenges and Solutions

6.5.1. Challenge 1: Manifest Schema Validation

Issue: Initial manifest validation failed:

The attribute 'Wide310x150Logo' on the element 'VisualElements' is not defined in the DTD/Schema.

Solution: Moved Wide310x150Logo to nested DefaultTile element:

```
<uap:VisualElements ...>
  <uap:DefaultTile Wide310x150Logo="Assets\LAN Discovery.png" />
</uap:VisualElements>
```

6.5.2. Challenge 2: Missing Assets in MSIX Package

Issue: Partner Center validation:

Package acceptance validation error: The following image(s) specified in the appxManifest.xml were not found:
Resources\chess_logo.png, Resources\AI Thinking.png, ...

Solution:

1. Created proper Assets folder structure
2. Copied all 4 images to ChessInstaller/MSIX/Assets/
3. Updated manifest to reference Assets\ instead of Resources\
4. Reopened MSIX in Packaging Tool and added assets manually

6.5.3. Challenge 3: Missing .NET runtimes

Issue: app can't run because the destination machine don't have .NET installed

Solution: run

6.6. Analysis

Successfully completed Microsoft Store deployment demonstrating:

Technical Competency:

- MSIX package creation and manifest configuration
- Understanding of Windows 10 app packaging requirements
- Asset management and branding for Store presence
- Compliance testing with WACK

Professional Publishing:

- Partner Center account registration and navigation
- Store listing optimization (description, screenshots, metadata)
- Privacy policy creation
- Restricted capabilities justification

Problem Solving:

- Debugged 4 major validation errors
- Applied schema knowledge to fix manifest issues
- Managed assets across different packaging stages

Project Management:

- Followed complete publishing workflow
- Documented certification timeline
- Prepared for public distribution

This deployment represents production-grade software distribution beyond academic exercises, providing real-world experience in:

- Cloud distribution platforms
- Store certification processes
- Professional branding and marketing
- Compliance and security requirements

7. Conclusion**7.1. Achievements Summary**

This deployment project successfully completed all four task levels:

Task 1.1 (Pass): Created basic WiX installer following walkthroughs

Task 1.2 (Credit): Deployed production C# chess application with 62.94 MB of resources

Task 1.3 (Distinction): Packaged 3 managed DLLs + 2 config files + native dependencies

Task 1.4 (High Distinction): Published to Microsoft Store with MSIX conversion and Partner Center submission

7.2. Final Remarks

This deployment demonstrates comprehensive Windows application distribution from development through public release, successfully navigating desktop installer creation, multi-DLL dependency management, cloud platform publishing, and store certification requirements. All tasks completed to High Distinction standard.

8. Appendices

8.1. Appendix A: Public Access

Microsoft Store:

<https://apps.microsoft.com/detail/9NSGRLLH9K7Z?hl=en-us&gl=VN&ocid=pdpshare>

GitHub Repository

<https://github.com/caphefalumi/Chess.NET>